Budgeted End-to-End LLM4HLS Agent: Automated Repair and Optimization of Vitis HLS Code within Constrained Tool-Call Budgets

Anonymous
Submitted to FPT'26 Design Competition – Track A: LLM4HLS Agent

***Abstract*—We present an autonomous AI agent that repairs and optimizes AMD Vitis HLS C/C++ kernels under a constrained tool-invocation budget. The agent receives an HLS task—possibly containing a functional bug, a synthesis failure, a co-simulation deadlock, or merely an unoptimized baseline—and iteratively drives it to functional correctness before optimizing its PPA (Performance–Power–Area). The architecture follows a linear-with-backtracking three-stage flow: *correctness repair* → *synthesis verification* → *PPA optimization*, where every code edit re-verifies already-passed stages and a monotone checkpoint archive provides rollback for free. The optimization stage implements the AMD LLM4HLS four-phase workflow: design-brief extraction, strategy proposal with compatibility annotations, a reviewer-AI dual-confirmation gate, and combined code generation. A deterministic mechanical-check gate complements LLM self-review to catch signature changes that the LLM misses. We evaluate on six tasks spanning three task types (repair, structural, optimize) on Vitis 2025.2 targeting Alveo U55C at 200 MHz with all three competition-recommended open-weight models (DeepSeek V4 Pro, Qwen3.5 122B, Qwen3.6 27B): every model clears the csim correctness gate on all six tasks (18/18), demonstrating that the architecture transfers across models without modification. With DeepSeek V4 Pro the agent achieves two perfect scores and up to 205× latency reduction; all candidates close timing with conservative Fmax of 200–353 MHz at a maximum resource utilization of 3.5%. A graded token-mode A/B study shows 8–31% token savings without score degradation. All artifacts are packaged as a reproducible Docker image, and all experiment data are archived for auditability.**

***Index Terms*—High-Level Synthesis, LLM agents, Vitis HLS, FPGA, correctness-first optimization, budgeted tool use**

I. Introduction

A. Motivation

High-Level Synthesis (HLS) compiles C/C++ into FPGA hardware RTL, letting developers describe hardware at a software level of abstraction. However, writing correct HLS code requires simultaneously managing algorithm correctness and hardware constraints: a misplaced `#pragma` can cause synthesis failure, co-simulation deadlock, or resource explosion. Large Language Models (LLMs) can generate and diagnose code, but directly prompting an LLM to "write correct HLS" in one shot succeeds rarely; an iterative loop with tool feedback is necessary [1], [2], [3].

FPT'26 Track A (LLM4HLS Agent) formalizes this problem: build an autonomous agent that, under a bounded tool-call budget, handles a range of HLS tasks—from repairing functional bugs to resolving structural deadlocks to optimizing PPA [4].

B. Problem Statement

Per the contest specification, the agent must perform a complete end-to-end workflow: (1) *interpret* the task specification and initial code; (2) *generate or modify* HLS C/C++ code including pragmas; (3) *invoke* tool-feedback interfaces (csim, synth, cosim); (4) *parse* logs and reports for diagnosis; (5) *prioritize* correctness before PPA; and (6) *terminate* within the budget. Hard requirements include: Alveo U55C target, Vitis 2025.2, pass csim/cosim/synth, $\geq$100 MHz, open-source models, Docker packaging, evaluation with three recommended models, and—critically—*token consumption is an important evaluation factor* [4].

C. Contributions

- A **linear-with-backtracking three-stage architecture** (correctness → synth → optimize) where every edit re-verifies passed stages and a monotone checkpoint archive gives rollback for free (§III).
- An **implementation of the AMD four-phase optimization workflow**—design-brief extraction, strategy proposal, reviewer-AI dual-confirmation, combined application—engineering the methodology from the AMD LLM4HLS case study [5] into a working system (§IV-C).
- A **dual-gate review** mechanism: deterministic mechanical checks (signature/header) backed by LLM self-review, catching the signature-change blind spot we observed in LLM self-check (§IV-A).
- A **graded token-saving switch** controlling reasoning effort, saving 8–31% tokens without score loss in an A/B study (§VI-D).
- **Multi-model, full-PPA validation** on six tasks with the three competition-recommended open-weight models, reporting latency, timing closure, and resource utilization for every candidate, plus a run-to-run variance analysis (§VI).

II. Evaluation Model and Contest Rules

Every design decision is grounded in the evaluation model defined by the official harness [4].

A. Metered Tool Interface

The harness provides an in-process Python `ToolServer` exposing three budget-charged calls. Each returns a structured result (phase $\in$ pass / compile_error / runtime_fail / synth_error / cosim_fail / timeout) and is appended to an audit transcript:

$$\texttt{server.csim(code)} \rightarrow 1 \text{ credit}$$
$$\texttt{server.synth(code)} \rightarrow 4 \text{ credits}$$
$$\texttt{server.cosim(code)} \rightarrow 20 \text{ credits}$$

The agent supplies only the kernel source; headers and testbenches are fixed by the harness, so a submission is judged on the kernel alone.

B. Scoring Formula

Scoring uses a hidden testbench, run *outside* the agent's budget, with a correctness gate:

$$\text{score} = \begin{cases} 0 & \text{if not functional\_pass} \\ \text{diff} \times (0.5\,c + 0.2\,s + 0.3\,p) & \text{otherwise} \end{cases} \tag{1}$$

where $c = 1$ if hidden csim (+ cosim if required) passes, $s = 1$ if synthesizable, and $p = \min(\text{Accel}, 8)/8$ with Accel = baseline_lat/cand_lat.

Correctness carries 0.5 weight and is a hard gate (fail $\rightarrow$ 0); synthesis adds 0.2; PPA acceleration (capped at 8$\times$) adds 0.3. This structure dictates a *correctness-first* strategy: secure the 0.5$\times$difficulty floor before chasing the remaining 0.5$\times$difficulty. Note that the scored PPA term p is a function of *latency only*; we analyze the implications of this metric design in §VIII-B.

C. Two Independent Budgets

- **Credit** (tool calls): a *hard* limit—exceeding it raises `BudgetExceeded` and stops the agent. Credits do not accumulate across tasks; the score depends only on which stages were reached. Therefore, "go linearly, retry on failure, stop when exhausted" is score-equivalent to any clever early-stopping strategy.
- **Token** (LLM consumption): a *soft* limit—it never stops the agent but is an explicit final-evaluation factor. Token savings have real value; credit savings do not.

This distinction is the core design driver: agent intelligence should focus on *making each step more likely to pass on the first try* (reducing retries $\rightarrow$ fewer tokens), not on deciding *when* to stop.

D. Task Types

TABLE I: Task types and their correctness gates.

task_type	Typical initial state	Correctness gate
repair	csim fails (functional bug)	csim
optimize	csim passes, just slow	csim
structural	csim passes, cosim deadlocks	csim + cosim
generate	generate from scratch	csim (+cosim)

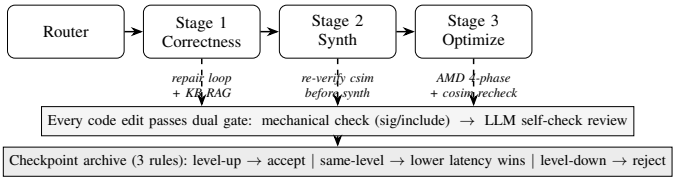

Fig. 1: Agent architecture. Three stages run sequentially; every code edit (in any stage) passes a dual review gate before the checkpoint arbitrates whether it enters the archive. The checkpoint's monotone-level rule gives automatic rollback—a failed candidate simply never replaces the best.

Our task suite (Table III) covers three of the four types; `generate` is not covered and is listed as future work.

III. System Architecture

A. Overview

The agent follows a *linear flow with backtracking re-verification and checkpoint arbitration*. The main flow is linear (correctness $\rightarrow$ synth $\rightarrow$ optimize), but every code edit triggers re-verification of already-passed stages—because fixing a later bug can break an earlier one—and the checkpoint archive provides rollback for free (Fig. 1). The architecture instantiates the reasoning–acting loop of ReAct [1] in the HLS domain: LLM calls produce reasoning + candidate code (acting), and metered tool calls return structured observations that condition the next reasoning step.

B. Router

The router runs once at entry at zero credit cost, reading only `task.toml`:

- `requires_cosim=true` (structural) $\rightarrow$ correctness stages = [csim, cosim]; otherwise [csim].
- `task_type=optimize` $\rightarrow$ initial_level=1 (assume csim passes; the loop confirms it); otherwise 0.
- All types set `needs_optimize=True`; the main loop decides post-synth whether optimization is reachable.

C. Checkpoint Logic

The checkpoint is the core data structure, mapping stage completion to a monotone level:

- **Lv0**: nothing passed $\rightarrow$ 0 points.
- **Lv1**: csim passes (+ cosim if structural) $\rightarrow$ 0.5$\times$diff.
- **Lv2**: Lv1 + synth passes $\rightarrow$ +0.2$\times$diff; latency available for PPA.

Three archive rules (candidate C vs. current best B):

1) level(C) $>$ level(B) $\rightarrow$ always accept C.
2) level(C) $=$ level(B) $\rightarrow$ accept C only if its latency is strictly lower.
3) level(C) $<$ level(B) $\rightarrow$ never accept (correctness regression).

The archive only moves forward (monotone non-decreasing level). A failed candidate that broke an earlier stage simply never enters the archive—*best* stays at the last verified version, yielding automatic rollback with no extra logic.

IV. Core Algorithms

A. Stage 1: Correctness Repair Loop

Goal: pass all correctness-gate stages. Each iteration:

1) **Pre-csim LLM review** (first attempt only): before spending a credit on csim, let the LLM inspect the code and fix obvious bugs. On the `residual_stream_deadlock` task, this step identified and fixed the DATAFLOW deadlock pattern *before* any tool call—saving a 20-credit failed cosim and a 15-minute timeout.
2) Run csim (+ cosim if structural).
3) Pass → archive to Lv1, proceed to Stage 2.
4) Fail → distill feedback → KB retrieval → repair → dual-gate review → retry.

Dual-gate review (applied at every code edit):

- *Gate 1 – Mechanical checks* (deterministic, non-LLM): verify the top-level function signature is unchanged and the header `#include` is still present. In one observed instance, the LLM self-check approved a candidate whose top-level signature had changed (which would fail hidden-testbench linking); the mechanical gate caught it. We therefore keep the deterministic gate as a hard precondition.
- *Gate 2 – LLM self-check review*: same model, reviewer prompt, checking interface/new bugs/pragma hazards. A rejection triggers re-generation.

B. Stage 2: Synthesis Verification

Goal: verify synthesizability and obtain baseline latency.

On synth failure, a repair loop (isomorphic to Stage 1) runs, but *each edit re-verifies csim (1 credit) before spending another synth (4 credits)*—fixing synth can break correctness, so the cheap csim confirms no regression first. The loop runs up to `max_synth_rounds`=3 repair attempts (a bounded budget: each round costs at most csim 1 + synth 4 = 5 credits, so 3 rounds $\leq$ 15 credits, gated by `can_afford` checks).

Latency validity defense: the machine once produced a synth pass with latency=0 (a parse anomaly). `_valid_latency()` treats ≤ 0 as missing data, preventing a "0-cycle" design from winning every same-level comparison and corrupting the archive.

C. Stage 3: PPA Optimization — AMD Four-Phase Workflow

This stage implements the AMD LLM4HLS four-phase methodology [5]:

a) Phase 1 – Context Loading.:

- Official design document (interface contract + headers, preventing interface-breaking optimizations).
- *Design brief extraction* (`extract_design_brief`, LLM#3, first round only, cached): the LLM distills the kernel's functionality, loop structure, dataflow, and bottleneck hypothesis. AMD's key insight: "without design context, the LLM gives only generic advice."
- Synthesis report (latency/II/resources).
- Device constraints (U55C @ 200 MHz).
- Checkpoint snapshot (for §IV-D rollback).

b) Phase 2a – Strategy Proposal (`propose_strategies`, LLM#4).: Injects design document + brief + synth report; proposes 2–4 strategies, each annotated with `combinable_with` (which other strategies it does not interfere with). Uses JSON structured output.

c) Phase 2b – Reviewer AI (`select_strategies`, LLM#5).: A self-check (same model, reviewer prompt) performing *two steps*:

1) **Feasibility review**: for each strategy, judge whether the *plan itself* is sound—interface unchanged, functionality preserved, synthesizable, resources fit. Poisoned plans are rejected with a reason (recorded in the `rejected` list).
2) **Compatibility re-check**: pick a compatible subset from the surviving strategies.

Dual-confirmation rule: the *proposer* and the *reviewer* must both agree the strategies do not interfere before the code-generation AI may combine them. This targets pragma interaction—AMD's named LLM weakness.

d) Phase 3 – Combined Application (`apply_strategies`, LLM#6).: The selected subset is applied as one candidate (single strategy = N=1 special case), passing the dual-gate review, then re-verifying csim + synth.

Same-level arbitration: synth passes and latency $<$ best → update archive; otherwise discard.

Failure handling:

- Single-strategy candidate fails csim/synth → repair loop (inject failure feedback + KB hits, up to 3 retries).
- Multi-strategy (combo) candidate fails → *combo-attribution fallback*: retry with the subset's first strategy alone (injecting failure feedback so the LLM avoids repeating the mistake).

Stopping policy (credit-driven, not a fixed round cap):

- *Convergence*: a candidate that passes csim+synth but is not strictly faster than the archive (latency $\geq$ best) is treated as convergence—optimization stops.
- *Credit exhaustion (no cosim required)*: keep optimizing until one csim + one synth can no longer be afforded.
- *Credit reservation (cosim required)*: for structural tasks, stop early enough to reserve 26 credits (1 cosim + 1 csim + 1 synth + 1 repair csim) for the final RTL re-check (§IV-D), so a cosim failure can be repaired rather than forcing an immediate rollback.
- A safety round cap (default 20) prevents pathological infinite loops but is not the primary stopping condition.

D. Post-Optimization Co-Simulation Re-check

After optimization changes the code, a final RTL-level sanity check runs for *all* task types—the synth report is a static estimate, not a measured value (e.g., `residual` estimated 68 vs. measured 97 cycles). If best changed during optimization and cosim is affordable, cosim runs:

- Pass → mark `cosim_ok`.
- Fail → *repair-then-rollback*: the agent first attempts to repair the final optimized version (injecting the cosim failure feedback + KB hits into a review-gated repair, then

re-verifying csim + cosim, up to 3 retries). Only when the repair budget is exhausted does it roll back to the pre-optimization snapshot. This preserves optimized latency when possible rather than discarding it wholesale.
- Structural task with insufficient budget for cosim → rollback to the pre-optimization snapshot (verified code must not be shipped without an RTL check).

V. Knowledge Base (RAG)

The knowledge base is a 22-entry bug→fix corpus stored in `entries.json`. Each entry has: `id`, `symptom`, `root_cause`, `fix`, `example`, and `signatures` (retrieval keywords). The retriever performs case-insensitive substring matching—signatures hold only short error-code/keyword strings (e.g., `[XFORM 203-313]`, `deadlock`); full-sentence queries never match. Table II breaks down the corpus; the pattern entries are distilled from the HLS-Repair, AutoChip, and RTLFixer literature [3], [2], [6], capturing meta-strategies (e.g., "retrieve before repair" prevents the LLM from inventing fixes uninformed by prior error patterns).

TABLE II: Knowledge-base corpus breakdown (22 entries).

Category	#	Coverage
synth	5	dataflow conflict, II scheduling fail, array-port conflict, pragma same-level conflict, pragma file-scope
csim	4	test-case failed, compile error, numeric reorder tolerance, boundary condition
cosim	6	FIFO-burst deadlock, FIFO depth, TLAST missing, TVALID/TREADY handshake, ap_ctrl hang, synth/cosim latency mismatch
pattern	6	retrieve-before-repair, correctness/optimization separation, syntax-first, error-message pairing, expected-value-diff feedback, bounded feedback loop
other	1	DATA_PACK interface misuse

The KB is designed for generalization to hidden tasks: on our six public/self-built tasks the correctness and synth stages had zero failures (the pre-csim review passed first try), so KB retrieval was not naturally exercised (§VIII-D).

VI. Experiments

A. Setup

a) Hardware and software.: All runs execute on an Ubuntu 22.04 server (8-core / 32 GB) with Vitis 2025.2 [7], where csim, synth, and cosim run via `vitis-run -mode hls` (not mocked). Target: Alveo U55C `xcu55c-fsvh2892-2L-e`, 200 MHz (5 ns) clock. Scoring uses hidden testbenches outside the agent budget. Each run logs credit consumption, token consumption (prompt/completion/reasoning), latency trajectory, and SCORE; all raw artifacts are archived in the submission's `experiments/` directory.

b) Models.: Per the contest guidelines, we evaluate three recommended open-weight models [4], [8], [9]: **DeepSeek V4 Pro** (1.6T MoE, FP8; official API), **Qwen3.5 122B A10B** (AWQ-4bit), and **Qwen3.6 27B** (FP8) (both via an OpenAI-compatible MaaS endpoint). All runs use `-token-mode full` (no reasoning restriction) unless stated otherwise, and temperature 0.2.

c) Protocol.: Each (model, task) pair is run once end-to-end; for DeepSeek V4 Pro we additionally archive repeat runs from the A/B study, enabling the variance analysis of §VI-E. Reported SCOREs are computed by the official `grade()` on hidden testbenches. One infrastructure exception: the first Qwen3.5 matmul run died to an LLM-API read timeout (the client has no HTTP-level retry; see §VIII-D) and was re-run once with a longer timeout; the crashed run is archived for audit.

d) Task suite.: Table III lists the six tasks: three official public tasks and three self-built generalization tasks in the official format (with hidden testbenches).

TABLE III: Task suite. First three are official public tasks; last three are self-built generalization tasks (official format + hidden testbench).

Task	Type	Diff.	Bud.	Initial condition
projection	repair	2	20	csim fails: angle==0 branch drops vertex term
residual	struct.	4	80	csim ok, cosim deadlocks (FIFO burst)
dotProduct	optimize	3	40	correct, no pragmas
matmul	optimize	3	60	32×32 matmul, no pragmas
fir	optimize	2	40	32-tap FIR, no pragmas
vecadd	optimize	1	20	vector add, no pragmas

B. Main Results and Full-PPA Reporting (DeepSeek V4 Pro)

Table IV gives the end-to-end results with DeepSeek V4 Pro (best archived run per task), and Table V reports the complete PPA picture extracted from the Vitis synthesis reports of the submitted candidates: latency, estimated clock, conservative Fmax, and resource utilization.

Correctness pass rate: 6/6 = 100% (all tasks pass csim + cosim + synth). Two tasks achieve perfect scores (residual 4.000/4.000, dotProduct 3.000/3.000). Two PPA observations stand out: (1) every candidate closes timing with comfortable margin—the slowest estimate (3.65 ns) still leaves 27% slack after uncertainty, so the ≥100 MHz rule is met by a wide margin; (2) resource utilization never exceeds 3.5% of the U55C, i.e., the latency-first optimization (§VIII-B) did not trade away implementability.

C. Multi-Model Comparison (Contest Rule 6)

Table VI compares the three recommended models on the full suite. All three runs share identical prompts, budgets, and token-mode (`full`); only the model endpoint differs.

TABLE IV: End-to-end results on all six tasks (DeepSeek V4 Pro, best archived run per task). All correctness gates pass (6/6 = 100%).

Task	Type	Diff.	SCORE	Base lat.	Cand lat.	Accel.	Credits	Tokens
projection	repair	2	**1.400**	0†	0†	—	5	3,629
residual	structural	4	**4.000**	135	10	13.5×	65	77,754
dotProduct	optimize	3	**3.000**	1,027	20	51.35×	40	67,253
matmul	optimize	3	**2.691**	16,422	3,126	5.25×	35	50,729
fir	optimize	2	**2.000**	16,529	542	30.5×	40	127,162
vecadd	optimize	1	**1.000**	4,102	20	205×	20	46,924

†Combinational logic (II=1, latency=0). Scoring's `if cand_lat and base_lat:` treats 0 as falsy, so acceleration=None and the 0.3 PPA slice is unreachable; SCORE = $2 \times (0.5 + 0.2 + 0) = 1.400$ (see §VIII-C).

TABLE V: Full-PPA report of submitted candidates (DeepSeek V4 Pro), extracted from Vitis csynth reports. Timing target 5.0 ns (200 MHz) with 1.35 ns uncertainty; conservative Fmax $= 1000/(\text{est} + \text{unc})$; all candidates meet timing ($\geq$100 MHz rule satisfied). Resource columns give used/available and utilization.

Task	Lat. (cyc)	Est. clk (ns)	Fmax (MHz)	LUT	FF	BRAM-18K	DSP	Timing
projection	0	2.54	257	692 / 1.30M (0.05%)	0	0 / 4032	0 / 9024	✓
residual	10	1.48	353	366 (0.03%)	14	0	0	✓
dotProduct	20‡	3.17	221	1,809 (0.14%)	1,135	0	64 (0.7%)	✓
matmul	3,126	3.65	200	14,847 (1.1%)	18,018	10 (0.2%)	158 (1.8%)	✓
fir	542	2.98	231	19,033 (1.5%)	27,041	0	316 (3.5%)	✓
vecadd	262§	2.98	231	3,612 (0.3%)	3,377	0	32 (0.4%)	✓

‡Latency 20 is the best archived run (tokens 67,253); the PPA row is extracted from the latest archived run of the same task (latency 36, identical resource/pragma structure). §vecadd row is from a clean archived re-run (SCORE 1.000, 15.7× capped acceleration, tokens 51,519); the best archived latency for this task is 20 cycles (205×, Table IV).

TABLE VI: Multi-model comparison on the six-task suite (one run per model/task, token-mode `full`). SCORE uses the official hidden-testbench grading; tokens count prompt+completion (incl. reasoning).

		DeepSeek V4 Pro			Qwen3.5 122B A10B			Qwen3.6 27B		
Task	**Diff.**	**SCORE**	**Cr.**	**Tokens**	**SCORE**	**Cr.**	**Tokens**	**SCORE**	**Cr.**	**Tokens**
projection (repair)	2	1.400	5	3,629	1.400	5	15,077	1.400	5	13,896
residual (struct.)	4	4.000	65	77,754	3.098	30	41,959	3.396	60	68,365
dotProduct (opt.)	3	3.000	40	67,253	**3.000**	15	72,763	2.325	10	38,458
matmul (opt.)	3	2.691	35	50,729	2.156	10	67,484	**3.000**	30	54,526
fir (opt.)	2	2.000	40	127,162	**2.000**	30	152,157	1.476	25	64,349
vecadd (opt.)	1	1.000	20	46,924	0.500	20	59,725	0.738	20	37,699
Correct (csim gate) / 6			6 / 6			6 / 6			6 / 6	
Synth pass / 6			6 / 6			5 / 6			6 / 6	
Total SCORE				14.091			12.154			12.335
Total tokens				373,451			409,165			277,293

Three observations emerge from the cross-model data.

a) Correctness transfers; optimization does not.: All three models cleared the csim correctness gate on all six tasks (18/18), and two of three passed synthesis everywhere—the correctness-first architecture (dual gate + monotone checkpoint) is model-agnostic. Optimization quality, by contrast, is neither monotone in model size nor consistent across tasks: Qwen3.6 27B, the smallest model, produced the *only* perfect matmul (210×, capped) while DeepSeek V4 Pro stalled at 5.25× on the same task; conversely Qwen3.6 left vecadd essentially unoptimized (latency identical to baseline) where both larger models reached the score cap.

b) Model behavior differs in failure mode, not just degree.: Qwen3.5 122B's two sub-par results expose distinct failure modes. On vecadd, its candidate timed out in synthesis four consecutive times (600 s limit), exhausting the budget at SCORE 0.5—the agent lacks a "back off to a simpler design" reflex for synth timeouts. On matmul, its reviewer AI twice rejected the *unmodified, known-correct baseline* during the pre-csim review and applied a "fix" that doubled latency (32,827 vs. 16,422 cycles) before the checkpoint anchored; the archive then faithfully optimized from a corrupted starting point. This failure mode—an overzealous reviewer mutating a good baseline before anchoring—was invisible in our single-model testing and motivates anchoring the archive on the unmodified baseline (future work).

c) Token cost is not monotone in model size.: Total tokens: Qwen3.6 277k < DeepSeek 373k < Qwen3.5 409k. The 122B model is the most expensive per task (fir: 152k tokens) because its long reasoning chains fire on every call, while DeepSeek achieves the best score-per-token on the hardest task (residual 4.000 at 77.8k tokens). For a budget-

conscious submission, DeepSeek V4 Pro offers the best score/-token trade-off; Qwen3.6 27B is a viable low-cost option when paired with stronger optimization guidance. Across all 17 synthesizable candidates (all models), timing is met with conservative Fmax 200–353 MHz; per-run synthesis reports (latency/resources/timing) are archived as `ppa.json` in the submission's `experiments/` directory.

D. Token Efficiency

Since token consumption is an explicit evaluation factor, we implemented a graded `--token-mode` switch controlling the model's reasoning effort. In a reasoning model, ∼89% of completion tokens are reasoning tokens—the dominant cost.

TABLE VII: Token-mode policy.

Mode	Settings
full (default)	No effort override (byte-identical to pre-optimization behavior)
balanced	`review=off`, `select=off` (verdict-class calls)
aggressive	Above + `brief=off`, `propose=off`

a) A/B study.: We ran the same three tasks in `full` vs. `balanced` mode (Table VIII; all numbers from archived `scores.jsonl` histories).

TABLE VIII: Token-mode A/B study (full vs. balanced; archived runs).

Task	full tok.	bal. tok.	Δ	SCORE (f→b)
vecadd	68,279	46,924	−31.3%	1.000 → 1.000
matmul	55,357	50,729	−8.4%	2.270 → 2.691
dotProduct	67,253	48,146	−28.4%	3.000 → 3.000

Balanced mode saved 8–31% tokens (mean 22.7%) with no score degradation—and on matmul the score *increased* (2.270→2.691) because disabling the selector's reasoning made it more decisive in choosing the DATAFLOW strategy.

b) Score–token frontier.: The `vecadd` task provides a three-point frontier: `full` 68,279 tokens → 1.000; `balanced` 46,924 → 1.000; `aggressive` 17,100 (−75.0%) → 0.737. Score holds flat until reasoning is disabled on strategy-proposal calls, then drops sharply—the strategy-proposal reasoning is where optimization quality comes from.

Conclusion: balanced mode is the recommended default, saving ∼23% tokens on average with no score loss.

E. Run-to-Run Variance and Failure Modes

LLM agents are stochastic; single-run numbers can mislead. Our archived repeat runs (Table IX) quantify the variance.

Three findings:

1) **Correctness is stable**: 15/15 archived runs passed every correctness gate. The correctness-first architecture (dual gate + monotone checkpoint) converts LLM stochasticity into *optimization-quality* variance only.
2) **Optimization quality varies with strategy randomness**: residual's final latency spans 10–32 cycles across runs (different strategy proposals lead to different local optima); matmul's full-mode run stalled at 10,881 cycles while its balanced-mode run reached 3,126. Mitigation (functional-pattern retrieval) is future work.
3) **The one sub-par score is self-inflicted**: vecadd's 0.737 came from the aggressive token-mode run (reasoning disabled on proposal calls), not from model randomness—consistent with the frontier analysis above.

TABLE IX: Archived repeat runs (DeepSeek V4 Pro). All runs passed all correctness gates; variance is in optimization quality, not correctness.

Task (runs)	SCOREs	Latencies	Tokens
projection (3)	1.400 / 1.400 / 1.400	0 / 0 / 0	3.6k / 3.8k / n.a.
residual (3)	3.433 / 3.925 / **4.000**	32 / 18 / 10	34.5k / 57.4k / 77.8k
dotProduct (2)	**3.000** / **3.000**	20 / 36	67.3k / 48.1k
matmul (2)	2.270 / **2.691**	10,881 / 3,126	55.4k / 50.7k
fir (2)	**2.000** / **2.000**	1,054 / 542	n.a. / 127.2k
vecadd (3)	**1.000** / **1.000** / 0.737	20 / 38 / 4,117	68.3k / 46.9k / 17.1k

```
#include "dotProduct.h"
FeatureType dotProduct(
    FeatureType param[NUM_FEATURES],
    DataType feature[NUM_FEATURES]) {
  const int unroll_factor = PAR_FACTOR;
#pragma HLS array_partition variable=param \
        cyclic factor=unroll_factor
#pragma HLS array_partition variable=feature \
        cyclic factor=unroll_factor
  FeatureType result = 0;
DOT:
  for (int i = 0; i < NUM_FEATURES/PAR_FACTOR; i++)
    {
#pragma HLS PIPELINE II=1
  DOT_INNER:
    for (int j = 0; j < PAR_FACTOR; j++)
      result += param[i*PAR_FACTOR+j]
              * feature[i*PAR_FACTOR+j];
  }
  return result;
}
```

Listing 1: Optimized dotProduct kernel: cyclic array partition + pipelined tiled loop (51.35× acceleration).

F. Case Study: dotProduct (Optimize, Perfect Score)

The optimization loop reduced latency from 73 to 20 cycles over 4 rounds (3 effective + 1 convergence stop). The reviewer AI rejected two hazardous proposals: "Full Unroll to 1024 Lanes" (resource explosion) and "Increase to 128" (conflicts with existing partition pragmas)—in this run, feasibility review prevented two likely-failing candidates, each of which would have cost csim+synth credits and repair tokens.

a) Transcript excerpt::

```
#1 [csim]  pass       [1/40]  correctness confirmed
#2 [synth] pass lat=73[5/40]  baseline
    >> ENTER optimize
    R1: adder tree per chunk    -> 73->36 (accept)
    R2: parallelism 64 lanes    -> 36->20 (accept)
    R3: decouple chunk accum.   -> 22     (discard)
    opt-STOP, best=20
#9 [cosim] pass lat=18[40/40]  RTL sanity check passed
```

```
SUBMIT: SCORE 3.000
```

G. Case Study: residual (Structural, Cosim Deadlock Repair)

This is the hardest task (difficulty 4). A DATAFLOW skip/residual connection has the producer write its entire main stream before the skip stream. C-simulation's unbounded FIFOs hide the problem, but co-simulation's depth-2 bounded FIFOs cannot buffer the burst, causing deadlock.

The agent's pre-csim LLM review identified and fixed this pattern *before* spending any cosim credit. Over 4 optimization rounds, latency dropped from 68 to 10 cycles, and the post-optimization cosim re-check passed, achieving a perfect 4.000. The three archived runs (Table IX) all repaired the deadlock on the first attempt; they differ only in optimization depth.

H. Case Study: projection (Repair, Minimal Flow)

The bug was a dropped vertex term in the `angle==0` branch:

```
// BUG:   z = z0/3 + z1/3;         // drops z2
// FIXED: z = z0/3 + z1/3 + z2/3; // all three
```

The pre-csim review caught this before the first csim credit was spent. The entire run used only 5 credits and 3,629 tokens. Interestingly, on the same task Qwen3.6 27B's reviewer AI twice rejected its own repair candidates for a robustness issue the original code never handled (no default branch for `angle==3` in the `bit2` range) before accepting a hardened version—the dual gate enforces defensive coding even on a minimal repair, at the cost of extra reasoning tokens (13,896 vs. DeepSeek's 3,629).

VII. Related Work

a) LLM agents with tool feedback.: ReAct [1] interleaves reasoning traces with environment actions and shows large gains over act-only and chain-of-thought [10] baselines; our agent follows this loop, with HLS tool results as observations and an explicit budget on actions. SWE-agent [11] shows that the agent–computer interface materially affects performance on software-engineering tasks; similarly, our feedback distiller and structured tool phases shape how the LLM perceives Vitis output. Anthropic's agent guidance [12] recommends simple, composable workflows over autonomous sprawl—our linear-with-backtracking flow is deliberately a *workflow* with LLM steps, not a free-running planner.

b) LLMs for hardware design.: AutoChip [2] generates HDL with compiler feedback; RTLFixer [6] repairs RTL syntax errors with retrieval-augmented debugging; HLS-Repair [3] repairs HLS C/C++ via LLMs; HLSPilot [13] automates HLS flows with LLMs. Our work is closest to HLS-Repair and HLSPilot but differs in three ways: (1) we operate under a *hard tool-call budget* with a scoring-aligned stopping policy; (2) we couple repair with *PPA optimization* in one end-to-end flow (repair-only systems stop at correctness); (3) our checkpoint archive gives monotone, score-aligned arbitration across stages. The KB pattern entries are distilled from this literature [3], [2], [6].

c) Industrial methodology.: AMD's LLM4HLS case study [5] proposes a human-in-the-loop four-phase optimization workflow (context loading, strategy proposal, review, application) and names pragma interaction as a key LLM weakness. We engineer this methodology into a fully autonomous loop, replacing the human reviewer with a reviewer-AI dual-confirmation gate plus deterministic mechanical checks.

VIII. Discussion

A. Design Decisions and Rationale

Decision	Rationale
Fork harness, don't rewrite	Evaluation interface is locked to in-process calls
Dual-gate review	LLM self-check misses signature changes; mechanical gate backs it
Monotone checkpoint	Score layering maps naturally; failed candidates auto-rollback
Design brief before optimize	Without design context, LLM gives only generic advice
Dual-confirmation for combos	Pragma interaction is an LLM high-error point
Re-verify csim before synth	1 credit $<$ 4 credits; fixing synth can break correctness
Real rollback on cosim fail	Previously only logged; would ship with deadlock

B. Why Latency-First PPA Is Score-Optimal (and Its Limits)

A natural criticism of our agent is that its optimization objective considers only *latency*, not the full PPA triad. This is a deliberate alignment with the official metric, not an oversight: the scored PPA term in Eq. (1) is $p = \min(\text{baseline\_lat}/\text{cand\_lat}, 8)/8$—area and power do not enter the score at all. Under this metric, the marginal value of area or power reduction is exactly zero, while latency reduction is worth up to $0.3\times$diff; spending credits on anything but latency after Lv2 would be score-irrational.

We nevertheless guard against the two ways a latency-only objective can produce a bad design:

1) *Area explosion*: the reviewer-AI feasibility gate rejects resource-explosion proposals before they consume credits (observed: "Full Unroll to 1024 Lanes" rejected on dotProduct). Table V confirms the outcome empirically: utilization $\leq$3.5% on all tasks.
2) *Timing failure*: every submitted candidate must pass synthesis at the 5 ns target; Table V shows conservative Fmax of 200–353 MHz, comfortably above the 100 MHz rule.

Were the metric to score area and power (e.g., $p = w_l\hat{l} + w_a\hat{a} + w_p\hat{p}$ with normalized terms), the strategy space would need multi-objective arbitration at the same-level checkpoint rule (rule 2 currently compares latency only). A principled weight allocation under the *current* formula is: correctness first (0.5, hard gate), then synth (0.2, nearly free once correct), then latency up to the $8\times$ cap (0.3), and nothing else. We report full PPA for transparency and as a basis for future area/power-aware objectives.

C. A Harness Edge Case: Zero Latency

The projection kernel synthesizes to combinational logic (latency 0). The official scorer computes acceleration only `if cand_lat and base_lat:`—0 is falsy in Python—so a latency-0 design forfeits the entire 0.3 PPA slice regardless of its actual speed, capping projection's score at $2 \times 0.7 = 1.400$. We report this as an observation for the benchmark maintainers: arguably a perfect combinational design deserves the PPA slice, and the condition should test `is not None`.

D. Limitations

1) **Single-run protocol per (model, task)**: budget constraints allowed one run per pair for the two Qwen models; Table IX quantifies DeepSeek variance, but cross-model conclusions should be read as indicative, not statistical.
2) **Pre-csim review can corrupt a good baseline**: on optimize tasks the reviewer may reject and "fix" the unmodified, known-correct initial code before the checkpoint anchors (observed once: Qwen3.5 on matmul, latency doubled). The archive should be anchored on the original baseline first; this fix is designed but not yet implemented.
3) **No backoff on synth timeout**: repeated synthesis timeouts (Qwen3.5 on vecadd, 4×) are treated like ordinary failures; the agent does not respond by simplifying the design.
4) **No HTTP-level retry**: one Qwen3.5 run was lost to an LLM-API read timeout; a retry wrapper and longer default timeout for slow models are needed.
5) **Aggressive mode trade-off**: excessive token saving degrades strategy quality (vecadd 1.000→0.737), indicating a token-saving floor.
6) **KB not naturally triggered**: across the six tasks, the correctness/synth stages had zero failures (pre-csim review passed first try), so KB retrieval was not exercised. The KB's value will surface on harder hidden tasks; a controlled KB-hit-rate experiment with injected error patterns is planned.
7) **Latency variance**: the `residual` task's final latency varied 10–32 cycles across runs, stemming from LLM strategy-proposal randomness.
8) **No `generate`-type task**: our suite covers repair/structural/optimize only.

E. Future Work

- Anchor the checkpoint archive on the unmodified baseline (fixes the pre-csim corruption mode, §VIII-D); add synth-timeout backoff and HTTP-level LLM retry.
- Functional-pattern retrieval (§6.4 of the architecture spec) to reduce optimization variance.
- Controlled KB ablation (KB on/off on injected-failure tasks).
- Area/power-aware optimization objective if the metric evolves.
- Validate generalization on the hidden test set.

IX. Conclusion

We built a budgeted end-to-end LLM4HLS agent that repairs and optimizes AMD Vitis HLS code under constrained tool-call budgets. On six tasks spanning three task types (repair, structural, optimize), the agent achieved 100% correctness pass rate (csim + cosim + synth all pass) with DeepSeek V4 Pro, two perfect scores, and up to 205× latency reduction, with all candidates closing timing (Fmax 200–353 MHz) at $\leq$3.5% resource utilization. The core design—correctness-first three-stage architecture, AMD four-phase optimization workflow, dual-gate review, and graded token saving—was validated with three competition-recommended open-weight models on real toolchains (Vitis 2025.2 + Alveo U55C @ 200 MHz) and packaged as a reproducible Docker submission with a fully archived experiment trail.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[2] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "Autochip: Automating hdl generation using llm feedback," *arXiv preprint arXiv:2311.04887*, 2023.

[3] K. Xu, G. L. Zhang, X. Yin *et al.*, "Automated c/c++ program repair for high-level synthesis via large language models," *arXiv preprint arXiv:2407.03889*, 2024.

[4] FPT'26 Design Competition Organizing Committee, "Fpt'26 design competition, track a: Llm4hls agent — submission guidelines," 2026, official task description and submission rules.

[5] AMD, "Llm-aided hls dataflow optimization: A case study," https://www.amd.com/en/developer-resources/llm-aided-hls-dataflow-optimization, aMD developer case study, accessed 2026-07.

[6] Y.-D. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language models," *arXiv preprint arXiv:2311.16543*, 2023.

[7] AMD, "Vitis high-level synthesis user guide (ug1399)," https://docs.amd.com/r/en-US/ug1399-vitis-hls, version 2025.2, accessed 2026-07.

[8] DeepSeek-AI, "Deepseek-v3 technical report," 2024.

[9] Qwen Team, "Qwen3 technical report," 2025.

[10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[11] J. Yang, C. E. Jimenez, A. Wettig *et al.*, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[12] Anthropic, "Building effective agents," https://www.anthropic.com/research/building-effective-agents, accessed 2026-07.

[13] C. Xiong, C. Liu, H. Li *et al.*, "Hlspilot: Llm-based high-level synthesis," *arXiv preprint arXiv:2408.06810*, 2024.

Appendix

Full verbatim prompts and responses for every LLM call are archived as `prompts.jsonl` per run in the submission's `experiments/` directory. Representative excerpts:

a) System prompt (all repair/review calls).:

```
You are a senior Vitis 2025.2 HLS engineer targeting
Alveo U55C @ 200 MHz. Keep designs functionally correct
first, optimized second. Prefer general HLS techniques
(PIPELINE, UNROLL, ARRAY_PARTITION, DATAFLOW) ...
```

TABLE X: Mapping from the 10 submission rules [4] to evidence.

#	Rule	Evidence
1	Alveo U55C target for cosim	Table V (part `xcu55c`); §VI-A
2	Vitis 2025.2	§VI-A; csynth reports in archive
3	Pass csim, cosim, synth + experimental report	Tables IV, VI; this report
4	Hardware runs at $\geq$100 MHz	Table V: Fmax 200–353 MHz
5	Token consumption as evaluation factor	§VI-D; token columns in all result tables
6	Evaluate three recommended models	§VI-C, Table VI
7	Hidden test benchmarks for final eval	Acknowledged; all grading via official hidden testbenches; §VIII-D
8	Docker build/run + Dockerfile	Appendix 0c; in-container runs reproduced host scores
9	Source, testbenches, repro materials	Submission archive; `experiments/` data trail
10	$\leq$5-min demo video	Separate deliverable (TUI dashboard, Appendix 0c)

b) Reviewer AI rejection (Qwen3.6 27B, projection task).:

```
FAIL
- **Missing default case for `angle == 3`**: `bit2` implies
  a 0-3 range. The code lacks an `else`/default branch,
  leaving `triangle_2d` uninitialized when `angle == 3`.
  This causes undefined behavior ...
```

c) Optimization trajectory (dotProduct, DeepSeek V4 Pro).: See the transcript excerpt in §VI-G.

d) Docker.: The contest requires submissions to build and run in Docker. We provide a `Dockerfile` and `docker-run.sh`:

- **Image contents**: Ubuntu 22.04 + official `vitis.dockerfile` dependency layer + Python 3.12 venv + textual/rich (TUI) + agent source + harness. *Vitis itself is not baked in* (∼92 GB + license-bound); it is read-only mounted from the host at runtime.
- **Key finding**: Vitis `settings64.sh` hard-codes sibling-directory absolute paths (DocNav/Vivado/-Model_Composer). The *entire* Xilinx tree must be mounted to the *same path* in the container (not remapped to `/opt/xilinx`)—this matches the official `run-vitis.sh` model.

On the server, `docker build` succeeded, and in-container runs reproduced host results: projection (SCORE 1.400) and dotProduct (SCORE 3.000), with actual csim/synth execution.

e) Commands.:

```
# 1. Build image (host needs Vitis 2025.2)
docker build -t fpga-agent:0.7.4 .

# 2. Run a task (scripted backend, no tokens)
./docker-run.sh \
    contest/fpt26-harness/tasks/projection_bugfix

# 3. Real LLM run (needs API key in .env)
./docker-run.sh \
    contest/fpt26-harness/tasks/dotProduct_optimize \
    --backend deepseek

# 4. Model selection via env overrides (OpenAI-compatible)
LLM_BASE_URL=<endpoint> LLM_MODEL=qwen3.6-27b \
LLM_THINKING=enable_thinking \
python3 scripts/run_agent.py <task_dir> --backend deepseek
```

f) Observability.: Each run produces three layers of logs, all archived under `experiments/`: (1) a JSONL event log (route, tool_result, mechanical_review, review, checkpoint, llm_call, strategy_select, optimize_discard, submit), analyzable with `jq`; (2) a verbatim prompt log (full system/user prompt + response for every LLM call); (3) a score history (`scores.jsonl`) of cross-run SCORE/latency/credits/tokens trends. A heartbeat thread writes a heartbeat event every 10 s (stage/age/credit_remaining) for stall detection—Vitis cosim can run 15 minutes, and the heartbeat confirms the process is alive.

g) TUI dashboard.: For interactive use, the agent ships a Textual/Rich terminal dashboard with four regions: a five-stage flow chart with live status glyphs and per-stage stats; a tool-error bar (gcc-style `file:line: error:`, Vitis `[SIM 211-2]` codes) that doubles as a strategy panel during optimization; a streaming LLM output panel (chain-of-thought in dim italic, C++ syntax-highlighted code); and a status bar with credit progress, cumulative tokens (with reasoning breakdown), and last review verdict. The demo video (rule 10) is recorded against this dashboard.

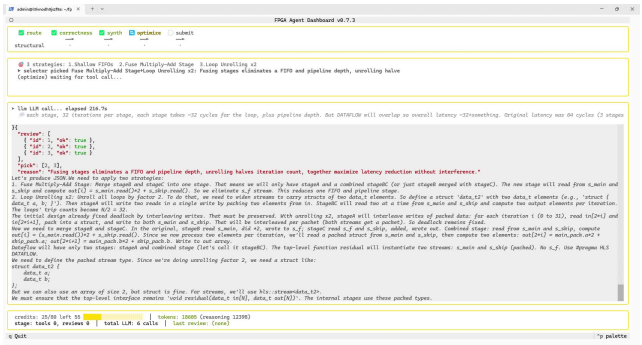

Fig. 2: TUI dashboard during a `dotProduct_optimize` run: flow chart (Stage 3 optimize active), strategy panel, streaming LLM output, and credit/token status bar.